Software Requirements Specification

# TeamFlow

## Collaborative Project and Task Management Platform

IT645 - Web and Mobile Development
Course Project (MERN Stack)

Dhirubhai Ambani University (DA-IICT)

Submitted to: **[Professor's Name - to be filled in]**


| Member | Roll No. | Functional Module |
|---|---|---|
| Bansari Akhani | 202512032 | Project & Workspace Management |
| Heer Sachdev | 202512117 | Task & Workflow Management |
| Ishpreet Kaur | 202512104 | Collaboration & Activity Tracking |
| Kriti Patel | 202512018 | Dashboard, Analytics & Notifications |


Document version 1.0 | Format: IEEE 830 / ISO/IEC/IEEE 29148 style | November 2026

# Document Control

## Revision History

| Version | Date | Author(s) | Description |
|---|---|---|---|
| 1.0 | November 2026 | All team members | Initial SRS submitted for project approval and development baseline. |

# Table of Contents

**1. Introduction** **5**

1.1 Purpose . . . 5

1.2 Scope . . . 5

1.3 Definitions, Acronyms and Abbreviations . . . 6

1.4 References . . . 6

1.5 Document Overview . . . 6

**2. Overall Description** **7**

2.1 Product Perspective . . . 7

2.2 Product Functions (summary) . . . 7

2.3 User Classes and Characteristics . . . 7

2.4 Operating Environment . . . 7

2.5 Design and Implementation Constraints . . . 7

2.6 Assumptions and Dependencies . . . 8

**3. Functional Requirements** **9**

3.1 Shared Core: Authentication and Common Infrastructure (all members) . . . 9

3.2 Module 1: Project and Workspace Management (Bansari Akhani, 202512032) . . . 9

3.3 Module 2: Task and Workflow Management (Heer Sachdev, 202512117) . . . 10

3.4 Module 3: Collaboration and Activity Tracking (Ishpreet Kaur, 202512104) . . . 10

3.5 Module 4: Dashboard, Analytics and Notifications (Kriti Patel, 202512018) . . . 11

**4. External Interface Requirements** **13**

4.1 User Interface (React Router pages) . . . 13

4.2 Hardware Interfaces . . . 13

4.3 Software Interfaces . . . 13

4.4 Communication Interfaces . . . 13

**5. REST API Specification** **14**

5.1 Authentication (Shared Core) . . . 14

5.2 Module 1: Projects and Members (Bansari Akhani) . . . 14

5.3 Module 2: Tasks (Heer Sachdev) . . . 14

5.4 Module 3: Comments and Activity (Ishpreet Kaur) . . . 15

5.5 Module 4: Dashboard and Notifications (Kriti Patel) . . . 15

5.6 HTTP Method Coverage . . . 15

5.7 Status Codes Used . . . 16

5.8 Response Formats . . . 16

**6. Data Requirements (Mongoose Models)** **17**

6.1 User (models/User.js) . . . 17

6.2 Project (models/Project.js) . . . 17

6.3 Task (models/Task.js) . . . 17
6.4 Comment (models/Comment.js) . . . 17
6.5 Activity (models/Activity.js) . . . 18
6.6 Notification (models/Notification.js) . . . 18
6.7 Relationships . . . 18
**7. Non-Functional Requirements 19**
**8. Architecture Requirements 20**
8.1 Backend Middleware Stack . . . 20
8.2 Backend Folder Structure . . . 20
8.3 Frontend Folder Structure and Hooks . . . 20
8.4 Environment Variables . . . 21
**9. Testing Requirements 22**
9.1 Shared Core / Authentication . . . 22
9.2 Module 1: Projects (Bansari Akhani) . . . 22
9.3 Module 2: Tasks (Heer Sachdev) . . . 22
9.4 Module 3: Collaboration (Ishpreet Kaur) . . . 23
9.5 Module 4: Dashboard and Notifications (Kriti Patel) . . . 23
9.6 Frontend Verification . . . 23
**10. Deployment and Documentation Requirements 24**
10.1 Deployment . . . 24
10.2 Repository and Documentation . . . 24
10.3 Individual Contribution Matrix . . . 24
**11. Traceability to IT645 Course Guidelines 25**
**Appendix A: Design Decisions for Team Confirmation 26**

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of **TeamFlow**, a web-based collaborative project and task management platform. It serves as the agreed baseline for design, implementation, testing and evaluation of the IT645 course project. It is written for the four team members, the course instructor and any evaluator who needs to verify that the delivered system meets the course project guidelines.

## 1.2 Scope

TeamFlow lets small groups, such as student project teams, organise their work in one place. Registered users create projects, add other registered users as members, create and assign tasks with priorities and deadlines, move tasks through a Kanban workflow (To Do, In Progress, Done), discuss tasks through comments, review an automatic activity history, and monitor progress through a dashboard. Email notifications are sent through the third-party **Resend** email Web API when a task is assigned, and on demand when a project owner sends a manual reminder.

**In scope:**

- JWT-based registration and login, and protected resources.
- Project and membership management with per-project roles (Project Owner, Member).
- Task CRUD, assignment, priority, due date, status workflow and query-based filtering.
- Task comments and an immutable activity history.
- Dashboards: totals, completion rate, overdue tasks and workload per member.
- Email on task assignment and a manual 'Send reminder' action via the Resend API.
- Deployment (Vercel, Render, MongoDB Atlas), Postman test collection, README and project report.

**Out of scope (not part of this version):**

- Real-time collaboration (WebSockets), chat, video calls.
- File attachments and avatar uploads; calendar synchronisation.
- Global administrator role or advanced role hierarchies.
- Automatic scheduled deadline emails (cron jobs); deadline email is sent only by manual reminder.
- Mobile application (React Native); the web application is responsive instead.
- Payments and billing.

## 1.3 Definitions, Acronyms and Abbreviations

| Term | Meaning |
|---|---|
| SRS | Software Requirements Specification. |
| MERN | MongoDB, Express.js, React, Node.js technology stack. |
| JWT | JSON Web Token used for stateless authentication. |
| REST | Representational State Transfer, the style used for the backend API. |
| CRUD | Create, Read, Update, Delete operations. |
| Project Owner | The user who created a project; has elevated rights within that project only. |
| Member | A registered user who has been added to a project. |
| Task | A unit of work belonging to exactly one project. |
| Overdue task | A task whose due date is earlier than the current date/time and whose status is not Done. |
| Activity | An automatically recorded, read-only history entry describing a change. |
| Resend | Third-party transactional email Web API used for notifications. |
| FR / NFR | Functional / Non-functional requirement. |

## 1.4 References

- IT645 Course Project Guidelines (course instructor).
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018, Systems and software engineering - Requirements engineering.
- Official documentation: React, React Router, Tailwind CSS, Node.js, Express, MongoDB, Mongoose, Resend.

## 1.5 Document Overview

Section 2 describes the product in general terms. Section 3 lists the functional requirements, organised by the four individual modules plus the shared authentication core. Section 4 specifies external interfaces. Section 5 specifies the REST API. Section 6 defines data requirements. Section 7 lists non-functional requirements. Section 8 describes backend and frontend architecture constraints. Section 9 defines the test plan. Section 10 covers deployment and documentation. Section 11 maps every course guideline to this SRS. Appendix A lists the design decisions that the team should confirm.

# 2. Overall Description

## 2.1 Product Perspective

TeamFlow is a new, self-contained three-tier web application. A React single-page application (SPA) communicates with a Node.js/Express REST API over HTTPS using JSON. The API persists data in MongoDB through Mongoose and calls the Resend email Web API from the server so that the API key is never exposed to the browser.

| Tier | Technology | Hosting (planned) | Responsibility |
|---|---|---|---|
| Presentation | React (functional components, hooks), React Router, Tailwind CSS | Vercel | UI, routing, form validation, loading/success/error states, HTTP calls via fetch()/axios. |
| Application | Node.js, Express.js, JWT, Mongoose | Render | REST API, authentication, authorisation, business rules, email integration. |
| Data | MongoDB with Mongoose schemas | MongoDB Atlas | Persistent storage of users, projects, tasks, comments, activities, notifications. |
| External | Resend email Web API | Resend cloud | Delivers transactional emails (task assignment, manual reminders). |

## 2.2 Product Functions (summary)

| Module | Owner | Summary |
|---|---|---|
| Shared Core | All members | Registration, login, JWT middleware, error handling, shared UI components, deployment, documentation. |
| M1 Project & Workspace | Bansari Akhani | Projects, membership, project-level access control, user search. |
| M2 Task & Workflow | Heer Sachdev | Tasks, assignment, priorities, deadlines, status workflow, filtering, Kanban board. |
| M3 Collaboration & Activity | Ishpreet Kaur | Task comments, activity logging service and history timelines. |
| M4 Dashboard, Analytics & Notifications | Kriti Patel | Progress dashboards, overdue tracking, workload aggregation, Resend email integration, notification log. |

## 2.3 User Classes and Characteristics

| User class | Description | Rights |
|---|---|---|
| Visitor (unauthenticated) | Anyone who has not logged in. | Can open Register and Login pages only. All other pages and API resources require a valid JWT. |
| Member | Authenticated user who belongs to a project. | In that project: view project, view/create/edit tasks, change task status and assignment, comment, view activity and dashboard, leave the project. Delete own comments; delete tasks they created. |
| Project Owner | Authenticated user who created the project. Roles are per project; the same person can be Owner in one project and Member in another. | All Member rights plus: edit/delete the project, add/remove members, delete any task or comment in the project, send manual reminders, view the notification log. |

## 2.4 Operating Environment

- Client: latest versions of Chrome, Edge, Firefox and Safari on desktop and mobile; viewport widths from 360 px upwards.
- Server: Node.js LTS runtime (v18 or later) on Render; MongoDB Atlas cluster.
- Network: HTTPS in production; CORS restricted to the deployed frontend origin.

## 2.5 Design and Implementation Constraints

- The system shall be built with the MERN stack (course requirement).
- UI shall use React functional components and hooks, React Router and Tailwind CSS.
- Backend shall follow a modular structure (routes, controllers, models, middleware) with schemas in **/models**.
- Secrets (database URI, JWT secret, Resend API key) shall be kept in **.env**, which is listed in **.gitignore**.
- Code shall follow ES6+ standards, use const/let and arrow functions, camelCase for variables/functions and PascalCase for React components.
- The third-party email API is Resend. Resend's test sender may only deliver to the account owner's own address until a sending domain is verified (see Appendix A, item D11). The demonstration plan must account for this.

## 2.6 Assumptions and Dependencies

- Users have a modern browser and an internet connection.
- Every member to be added to a project has already registered with a unique email address.
- The Resend service, MongoDB Atlas, Render and Vercel are available during development and evaluation.
- Free-tier hosting may have cold starts; the team will open the backend before the demonstration.

# 3. Functional Requirements

Each module below includes its own frontend, backend and testing requirements, as required by the course guidelines. Priority **Must** means the feature is required for the final demonstration; **Should** means it is planned but may be simplified if time is short.

## 3.1 Shared Core: Authentication and Common Infrastructure (all members)

Built jointly so that every member understands the login flow, token verification and shared conventions. Each member still owns and tests the features of their own module.

| ID | Requirement | Priority |
|---|---|---|
| FR-AUTH-01 | A visitor shall register with name, email and password. Email must be valid and unique (case-insensitive); password must be at least 8 characters. The password shall be stored only as a bcrypt hash and never returned by any API. | Must |
| FR-AUTH-02 | Registration with a duplicate email shall fail with 409. Invalid or missing fields shall fail with 400 and a field-level error list. | Must |
| FR-AUTH-03 | A user shall log in with email and password. On success the API returns a signed JWT and the user profile. Wrong credentials return 401 with a generic message (no hint which field was wrong). | Must |
| FR-AUTH-04 | The JWT shall be signed with JWT_SECRET, expire after JWT_EXPIRES_IN, and be sent by the client in the Authorization header as 'Bearer <token>'. | Must |
| FR-AUTH-05 | A reusable authentication middleware shall protect all non-auth routes and return 401 for missing, malformed, invalid or expired tokens. | Must |
| FR-AUTH-06 | A logged-in user shall be able to fetch their own profile through GET /api/auth/me. | Must |
| FR-AUTH-07 | The frontend shall keep authentication state in a React Context (AuthContext), persist the token across page reloads, provide a logout action that clears it, and redirect unauthenticated visitors to /login through a ProtectedRoute component. | Must |
| FR-COM-01 | All API responses shall use one consistent JSON envelope: success responses contain *success: true*, *data* and optional *message*/*pagination*; error responses contain *success: false*, *message* and optional *errors[]*. | Must |
| FR-COM-02 | Centralised error-handling middleware shall convert validation errors, invalid ObjectIds, duplicate keys, authentication errors and unexpected failures to the appropriate HTTP status codes without leaking stack traces in production. | Must |
| FR-COM-03 | Shared reusable frontend components shall exist for: Navbar, ProtectedRoute, Spinner, ErrorMessage/Alert, Toast, Modal, ConfirmDialog, Pagination, EmptyState, StatusBadge and PriorityBadge. | Must |

## 3.2 Module 1: Project and Workspace Management (Bansari Akhani, 202512032)

**Goal:** let users create projects, control who belongs to them, and guarantee that project data is visible only to its members.

| ID | Requirement | Priority |
|---|---|---|
| FR-PRJ-01 | A logged-in user shall create a project with name (required, 3-80 chars), description (optional, up to 500 chars), start date and end date (optional; end date must not be before start date). The creator becomes Project Owner and the first member. Response 201. | Must |
| FR-PRJ-02 | A user shall list only the projects they belong to. Query parameters: *search* (name), *sort* (createdAt, name, endDate; prefix '-' for descending), *page* and *limit* (default 10, maximum 50). The response includes pagination metadata. | Must |
| FR-PRJ-03 | A member shall view project details including the populated member list (name, email) and the identity of the Owner. A non-member receives 403; a non-existent project receives 404; a malformed id receives 400. | Must |
| FR-PRJ-04 | The Owner shall update a project with a full replacement of editable fields (name, description, startDate, endDate) using PUT. A Member receives 403. | Must |
| FR-PRJ-05 | The Owner shall delete a project. Deletion shall cascade to the project's tasks, comments, activities and notifications. | Must |

| ID | Requirement | Priority |
|---|---|---|
| FR-PRJ-06 | The Owner shall add a member by email. The user must be registered (404 otherwise) and not already a member (409). | Must |
| FR-PRJ-07 | The Owner shall remove a member; a member shall be able to leave a project themselves. The Owner cannot be removed (400). Tasks assigned to a removed member become unassigned. | Must |
| FR-PRJ-08 | Authenticated users shall search registered users by name or email (minimum 2 characters, maximum 10 results, returning only id, name and email) to help the Owner add members. | Should |
| FR-PRJ-09 | Reusable middleware *checkProjectMember* and *checkProjectOwner* shall be provided and used by every project-scoped route in all modules. | Must |
| FR-PRJ-10 | Membership changes shall be recorded through the activity-logging service of Module 3 (MEMBER_ADDED, MEMBER_REMOVED). | Must |
| FR-PRJ-11 | **Frontend:** Projects list page (search, sort, pagination), create/edit form, project details page with member list and add-member control, and a confirmation dialog before delete/remove. All requests show loading, success and error states. | Must |
| FR-PRJ-12 | **Testing:** Postman requests for create/list/get/update/delete project and add/remove member covering success, validation failure, unauthenticated access, non-member access, non-owner modification and unknown ids. | Must |

## 3.3 Module 2: Task and Workflow Management (Heer Sachdev, 202512117)

**Goal:** manage the complete lifecycle of a task from creation to completion.

| ID | Requirement | Priority |
|---|---|---|
| FR-TSK-01 | A project member shall create a task with title (required, 3-100 chars), description (optional, up to 1000 chars), priority (Low, Medium, High; default Medium) and due date (optional, valid date). Initial status is 'To Do'. Response 201. | Must |
| FR-TSK-02 | A task may be assigned to a user who is a member of the same project; assigning a non-member returns 400. A task may be unassigned. | Must |
| FR-TSK-03 | A member shall list tasks of a project with query parameters: *status*, *priority*, *assignee* (user id or 'me'), *dueFrom*, *dueTo*, *overdue=true*, *search* (title), *sort* (dueDate, priority, createdAt; '-' prefix for descending), *page*, *limit*. | Must |
| FR-TSK-04 | A member shall view one task by id, including populated assignee and creator names. | Must |
| FR-TSK-05 | A member shall update a task's editable fields (title, description, priority, dueDate) with PUT; all editable fields must be supplied. | Must |
| FR-TSK-06 | A member shall change task status with PATCH. Allowed statuses: To Do, In Progress, Done. Allowed transitions: To Do to In Progress; In Progress to Done; In Progress to To Do; Done to In Progress (reopen). Any other transition, an unknown status or the same status returns 400 with an explanatory message. | Must |
| FR-TSK-07 | A member shall change or clear a task's assignee with PATCH (separate from PUT). Assignment to a new assignee triggers the assignment email described in FR-DSH-06. | Must |
| FR-TSK-08 | The Project Owner or the task's creator shall delete a task; others receive 403. Comments of the deleted task are removed; its activity records are kept with a task-title snapshot. | Must |
| FR-TSK-09 | Task creation, edits, assignment changes, status changes and deletion shall be recorded through the activity-logging service (Module 3). | Must |
| FR-TSK-10 | **Frontend:** Kanban board with three status columns, TaskCard component, create/edit modal (title field focused with useRef), filter and sort bar, pagination or 'load more', a 'My tasks' filter, and a task details page. Disallowed status moves display the server's message. | Must |
| FR-TSK-11 | **Testing:** Postman requests for task create/list/get/update/status/assign/delete with valid and invalid payloads, each filter combination, invalid transitions, assigning a non-member, and unauthorised modification. | Must |

## 3.4 Module 3: Collaboration and Activity Tracking (Ishpreet Kaur, 202512104)

**Goal:** make discussions and changes on tasks visible and traceable.

| ID | Requirement | Priority |
|---|---|---|
| FR-COL-01 | A project member shall add a comment to a task. Text is required, trimmed, and 1-1000 characters. Response 201. | Must |
| FR-COL-02 | A project member shall list a task's comments with author name and timestamps, ordered oldest first by default (*order=asc\|desc*), with *page* and *limit*. | Must |
| FR-COL-03 | A comment's author shall edit their own comment with PUT; the comment is marked as edited with an editedAt time. Other users receive 403. | Must |
| FR-COL-04 | A comment's author or the Project Owner shall delete a comment. | Must |
| FR-COL-05 | A reusable *logActivity* service shall create an activity record (project, task, actor, action, details, timestamp). It shall be called by Modules 1, 2 and 3. Actions: TASK_CREATED, TASK_UPDATED, TASK_ASSIGNED, STATUS_CHANGED, TASK_DELETED, COMMENT_ADDED, COMMENT_DELETED, MEMBER_ADDED, MEMBER_REMOVED. | Must |
| FR-COL-06 | Members shall view the activity history of a task and of a whole project (newest first) with optional *action* filter and pagination. | Must |
| FR-COL-07 | Activity records are immutable: the API shall not provide update or delete endpoints for them; they are removed only by project deletion. | Must |
| FR-COL-08 | Comment and activity endpoints shall verify that the requester is a member of the project that owns the task (middleware that loads the task, then checks membership). | Must |
| FR-COL-09 | **Frontend:** CommentThread and CommentForm components (textarea focused with useRef, character counter, edit/delete shown only to permitted users), ActivityTimeline component with relative times, and a project activity feed page. | Must |
| FR-COL-10 | **Testing:** Postman requests for comment create/list/edit/delete (empty text, over-length text, non-member, non-author), and for activity creation after task/status/member actions, plus verification that activity cannot be modified. | Must |

## 3.5 Module 4: Dashboard, Analytics and Notifications (Kriti Patel, 202512018)

**Goal:** give teams visibility of progress and deliver email notifications through a third-party Web API.

| ID | Requirement | Priority |
|---|---|---|
| FR-DSH-01 | A project member shall view a project summary: total tasks, count per status, count per priority, completed count, overdue count, tasks due within the next N days (*dueSoonDays*, default 3) and completion rate (percentage; 0 when the project has no tasks). | Must |
| FR-DSH-02 | A project member shall view workload per member: assigned, To Do, In Progress, Done and overdue counts, plus an 'Unassigned' bucket. Calculated with a MongoDB aggregation pipeline. | Must |
| FR-DSH-03 | A project member shall list overdue tasks (due date earlier than now and status not Done), oldest due date first, with the number of days overdue. | Must |
| FR-DSH-04 | Summary and workload endpoints shall accept optional *dateFrom* and *dateTo* query parameters that filter by task due date. | Should |
| FR-DSH-05 | A user shall view a personal dashboard across all their projects: tasks assigned to them, overdue, and due within 7 days. | Should |
| FR-DSH-06 | When a task is created with an assignee, or its assignee is changed to a different user, the backend shall send an email to the new assignee through the Resend API. The email contains the task title, project name, priority, due date, a link to the task and a logo served from the static /public folder. If the email fails, the task operation shall still succeed and the response shall report the notification status. | Must |
| FR-DSH-07 | The Project Owner shall send a manual reminder for a task through POST. The task must have an assignee and must not be Done (400 otherwise). A Resend failure returns 502. | Must |
| FR-DSH-08 | Every email attempt shall be stored in a Notification record (type, recipient, task, triggeredBy, status sent/failed, provider message id, error text). The Owner can list notifications of a project with pagination. | Must |
| FR-DSH-09 | The Resend API key and sender address shall be read from environment variables (RESEND_API_KEY, EMAIL_FROM) only; the key shall never reach the browser or the repository. | Must |
| FR-DSH-10 | **Frontend:** Dashboard page with summary cards, a status/priority chart, a workload table or bar chart, an overdue list, date filters, a 'Send reminder' button with visible loading/success/error feedback, and a notification history table for Owners. Empty projects display zero values and an empty-state message. | Must |

| ID | Requirement | Priority |
|---|---|---|
| FR-DSH-11 | **Testing:** Postman requests verifying calculated values against known seed data, empty project, overdue rule boundaries, invalid date filters, non-member access, reminder on a Done or unassigned task, and email failure handling (e.g. invalid API key). | Must |

# 4. External Interface Requirements

## 4.1 User Interface (React Router pages)

| Route | Page | Access | Module |
|---|---|---|---|
| `/login` | Login form | Public | Shared |
| `/register` | Registration form | Public | Shared |
| `/dashboard` | Personal dashboard (my tasks, overdue, due soon) | Authenticated | M4 |
| `/projects` | Project list with search, sort, pagination, create project | Authenticated | M1 |
| `/projects/:projectId` | Project details, members, add/remove member, edit/delete project | Member (Owner for edits) | M1 |
| `/projects/:projectId/board` | Kanban board, filters, create/edit task | Member | M2 |
| `/projects/:projectId/tasks/:taskId` | Task details with comments and activity timeline | Member | M2 + M3 |
| `/projects/:projectId/activity` | Project activity feed | Member | M3 |
| `/projects/:projectId/dashboard` | Project dashboard, workload, overdue, notifications | Member (log: Owner) | M4 |
| * | 404 Not Found page | Public | Shared |

### UI requirements

- Pages shall use semantic HTML elements (header, nav, main, section, article, form, label, button, footer).
- Layout shall be responsive using Tailwind breakpoints: single column on mobile (360 px and above), two columns on tablet (768 px), full board and dashboard grid on desktop (1280 px).
- Every data-loading view shall show a loading indicator, an error message with a retry option, and an empty-state message when there is no data.
- Every mutating action shall give success or error feedback (toast or inline message) and disable its submit button while pending.
- Forms shall show field-level validation messages returned by the API.

## 4.2 Hardware Interfaces

None. The system runs in a browser and on cloud servers.

## 4.3 Software Interfaces

| Interface | Purpose | Notes |
|---|---|---|
| MongoDB Atlas via Mongoose | Persistent storage | Connection string in MONGO_URI (.env). |
| Resend REST API (POST https://api.resend.com/emails) | Send assignment and reminder emails | Called only from the backend with RESEND_API_KEY; sender in EMAIL_FROM; response id stored in Notification. |
| Browser fetch()/axios | Frontend to backend HTTP | Base URL from VITE_API_BASE_URL; token in Authorization header. |

## 4.4 Communication Interfaces

- HTTPS with JSON bodies (Content-Type: application/json) between client and API.
- CORS enabled only for the origin in CLIENT_URL (plus localhost during development).
- Static files from the backend **/public** directory (logo, icons, email assets) served with express.static() at the path /public.

# 5. REST API Specification

Base path: **/api**. All endpoints except register and login require the header **Authorization: Bearer <token>**. Resource ids in the path are MongoDB ObjectIds; a malformed id returns 400. Query parameters are used for filtering, searching, sorting and pagination only; resources are identified by route parameters.

## 5.1 Authentication (Shared Core)

| Method | Endpoint | Access | Description | OK | Errors |
|---|---|---|---|---|---|
| POST | `/api/auth/register` | Public | Create account | 201 | 400, 409 |
| POST | `/api/auth/login` | Public | Log in, receive JWT | 200 | 400, 401 |
| GET | `/api/auth/me` | Auth | Current user profile | 200 | 401 |

## 5.2 Module 1: Projects and Members (Bansari Akhani)

| Method | Endpoint | Access | Description | OK | Errors |
|---|---|---|---|---|---|
| POST | `/api/projects` | Auth | Create project (caller becomes Owner) | 201 | 400, 401 |
| GET | `/api/projects?search=&sort=&page=&limit=` | Auth | List my projects | 200 | 400, 401 |
| GET | `/api/projects/:projectId` | Member | Project details with members | 200 | 400, 401, 403, 404 |
| PUT | `/api/projects/:projectId` | Owner | Replace editable project fields | 200 | 400, 401, 403, 404 |
| DELETE | `/api/projects/:projectId` | Owner | Delete project and related data | 200 | 401, 403, 404 |
| POST | `/api/projects/:projectId/members` | Owner | Add member by email (body: email) | 201 | 400, 403, 404, 409 |
| DELETE | `/api/projects/:projectId/members/:userId` | Owner or self | Remove member / leave project | 200 | 400, 403, 404 |
| GET | `/api/users?search=` | Auth | Search registered users (id, name, email) | 200 | 400, 401 |

## 5.3 Module 2: Tasks (Heer Sachdev)

| Method | Endpoint | Access | Description | OK | Errors |
|---|---|---|---|---|---|
| POST | `/api/projects/:projectId/tasks` | Member | Create task | 201 | 400, 403, 404 |
| GET | `/api/projects/:projectId/tasks?status=&priority=&assignee=&dueFrom=&dueTo=&overdue=&search=&sort=&page=&limit=` | Member | List and filter tasks | 200 | 400, 403, 404 |
| GET | `/api/tasks/:taskId` | Member | Task details | 200 | 400, 403, 404 |
| PUT | `/api/tasks/:taskId` | Member | Replace title, description, priority, dueDate | 200 | 400, 403, 404 |
| PATCH | `/api/tasks/:taskId/status` | Member | Change status (body: status) | 200 | 400, 403, 404 |
| PATCH | `/api/tasks/:taskId/assignee` | Member | Assign/unassign (body: assignee or null) | 200 | 400, 403, 404 |
| DELETE | `/api/tasks/:taskId` | Owner or creator | Delete task | 200 | 403, 404 |

## 5.4 Module 3: Comments and Activity (Ishpreet Kaur)

| Method | Endpoint | Access | Description | OK | Errors |
|---|---|---|---|---|---|
| POST | `/api/tasks/:taskId/comments` | Member | Add comment (body: text) | 201 | 400, 403, 404 |
| GET | `/api/tasks/:taskId/comments?order=&page=&limit=` | Member | List comments | 200 | 400, 403, 404 |
| PUT | `/api/comments/:commentId` | Author | Edit comment text | 200 | 400, 403, 404 |
| DELETE | `/api/comments/:commentId` | Author or Owner | Delete comment | 200 | 403, 404 |
| GET | `/api/tasks/:taskId/activity?action=&page=&limit=` | Member | Task activity history | 200 | 400, 403, 404 |
| GET | `/api/projects/:projectId/activity?action=&page=&limit=` | Member | Project activity feed | 200 | 400, 403, 404 |

## 5.5 Module 4: Dashboard and Notifications (Kriti Patel)

| Method | Endpoint | Access | Description | OK | Errors |
|---|---|---|---|---|---|
| GET | `/api/projects/:projectId/dashboard/summary?dueSoonDays=&dateFrom=&dateTo=` | Member | Totals, status/priority counts, completion rate | 200 | 400, 403, 404 |
| GET | `/api/projects/:projectId/dashboard/workload?dateFrom=&dateTo=` | Member | Per-member workload (aggregation) | 200 | 400, 403, 404 |
| GET | `/api/projects/:projectId/dashboard/overdue` | Member | Overdue tasks with days overdue | 200 | 403, 404 |
| GET | `/api/dashboard/me` | Auth | Personal cross-project summary | 200 | 401 |
| POST | `/api/tasks/:taskId/remind` | Owner | Send manual reminder email via Resend | 200 | 400, 403, 404, 502 |
| GET | `/api/projects/:projectId/notifications?page=&limit=` | Owner | Email notification log | 200 | 403, 404 |

## 5.6 HTTP Method Coverage

| Method | Genuine use in TeamFlow |
|---|---|
| GET | Listing/reading projects, tasks, comments, activity, dashboards. |
| POST | Register, login, create project/task/comment, add member, send reminder. |
| PUT | Replace project fields, task fields, and comment text. |
| PATCH | Partial change of task status and task assignee. |
| DELETE | Delete project, task, comment; remove member. |

## 5.7 Status Codes Used

| Code | Meaning | Used when |
|---|---|---|
| 200 OK | Success | Successful read, update, delete. |
| 201 Created | Resource created | Register, create project/task/comment, add member. |
| 400 Bad Request | Invalid input | Validation failure, malformed id, invalid status transition, assigning non-member, reminder on invalid task. |
| 401 Unauthorized | Not authenticated | Missing/invalid/expired token, wrong login credentials. |
| 403 Forbidden | Not permitted | Not a project member, not the Owner/author when required. |
| 404 Not Found | Missing resource | Unknown project, task, comment, user or route. |
| 409 Conflict | Duplicate | Email already registered, user already a member. |
| 500 Internal Server Error | Unexpected failure | Unhandled server or database error. |
| 502 Bad Gateway | Third-party failure | Resend API call failed during a manual reminder. |

## 5.8 Response Formats

```
// Success (list)
{ "success": true, "data": [ ... ], "pagination": { "page": 1, "limit": 10, "total": 37, "pages": 4 } }

// Error
{ "success": false, "message": "Validation failed",
  "errors": [ { "field": "title", "message": "Title must be 3-100 characters" } ] }
```

# 6. Data Requirements (Mongoose Models)

All schemas are defined in the **/models** directory with validation, *timestamps: true* and the indexes listed. Relationships use ObjectId references.

## 6.1 User (models/User.js)

| Field | Type | Rules |
|---|---|---|
| name | String | required, trimmed, 2-60 chars |
| email | String | required, unique, lowercase, valid email format |
| password | String | required, bcrypt hash, *select: false* |
| createdAt / updatedAt | Date | automatic |

## 6.2 Project (models/Project.js)

| Field | Type | Rules |
|---|---|---|
| name | String | required, 3-80 chars |
| description | String | optional, max 500 |
| owner | ObjectId (User) | required |
| members | [{ user: ObjectId (User), joinedAt: Date }] | contains the owner; no duplicate users |
| startDate / endDate | Date | optional; endDate must not be before startDate |
| Indexes | - | members.user; owner |

## 6.3 Task (models/Task.js)

| Field | Type | Rules |
|---|---|---|
| project | ObjectId (Project) | required |
| title | String | required, 3-100 chars |
| description | String | optional, max 1000 |
| status | String enum | To Do \| In Progress \| Done; default To Do |
| priority | String enum | Low \| Medium \| High; default Medium |
| dueDate | Date | optional |
| assignee | ObjectId (User) or null | must be a project member |
| createdBy | ObjectId (User) | required |
| Indexes | - | {project, status}, {project, assignee}, {project, dueDate} |

## 6.4 Comment (models/Comment.js)

| Field | Type | Rules |
|---|---|---|
| task | ObjectId (Task) | required |
| project | ObjectId (Project) | required (denormalised for fast access checks) |
| author | ObjectId (User) | required |
| text | String | required, trimmed, 1-1000 chars |
| editedAt | Date | set when edited |
| Indexes | - | {task, createdAt} |

## 6.5 Activity (models/Activity.js)

| Field | Type | Rules |
|---|---|---|
| project | ObjectId (Project) | required |
| task | ObjectId (Task) or null | null for project-level events |
| taskTitle | String | snapshot kept after task deletion |
| actor | ObjectId (User) | required |
| action | String enum | TASK_CREATED, TASK_UPDATED, TASK_ASSIGNED, STATUS_CHANGED, TASK_DELETED, COMMENT_ADDED, COMMENT_DELETED, MEMBER_ADDED, MEMBER_REMOVED |
| details | Object | e.g. { field, from, to } |
| Indexes | - | {project, createdAt: -1}, {task, createdAt: -1} |

## 6.6 Notification (models/Notification.js)

| Field | Type | Rules |
|---|---|---|
| project, task | ObjectId | required |
| recipient | ObjectId (User) | required |
| triggeredBy | ObjectId (User) | required |
| type | String enum | TASK_ASSIGNED \| TASK_REMINDER |
| status | String enum | sent \| failed |
| providerMessageId | String | Resend message id when sent |
| error | String | error text when failed |

## 6.7 Relationships

```
User 1---* Project (owner)        User *---* Project (members)
Project 1---* Task               User 1---* Task (assignee, createdBy)
Task 1---* Comment               Task 1---* Activity      Project 1---* Notification
Deleting a project removes its tasks, comments, activities and notifications.
Deleting a task removes its comments; its activities remain with taskTitle.
```

# 7. Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| NFR-01 | Performance | Under normal load, list and read endpoints should respond within 500 ms (excluding cold start and third-party calls) for projects with up to 1,000 tasks. Pages should become interactive within 3 seconds on a broadband connection. |
| NFR-02 | Security | Passwords hashed with bcrypt; JWT secret and all keys only in environment variables; .env ignored by Git; HTTPS in production; CORS limited to the frontend origin; all inputs validated server-side; ObjectIds validated before queries. |
| NFR-03 | Authorisation | Every project-scoped request verifies membership on the server; the frontend hiding a button is never the only protection. |
| NFR-04 | Reliability | Unhandled errors are caught by central middleware and returned as JSON with proper status codes; a failed email never aborts the primary task operation. |
| NFR-05 | Usability | Consistent navigation, clear validation messages, confirmation before destructive actions, responsive from 360 px to desktop. |
| NFR-06 | Accessibility | Semantic HTML, labelled form controls, sufficient colour contrast, keyboard-operable buttons and dialogs. |
| NFR-07 | Maintainability | Modular folders, ES6+ syntax, const/let, arrow functions, camelCase for variables/functions, PascalCase for components, no duplicated logic (shared middleware, services and components). |
| NFR-08 | Portability | Runs on Node.js LTS (v18+) with configuration only through environment variables. |
| NFR-09 | Testability | All major endpoints covered by a Postman collection with automated assertions; seed data available for dashboard verification. |
| NFR-10 | Documentation | README, .env.example and a short report with screenshots are maintained in the GitHub repository. |

# 8. Architecture Requirements

## 8.1 Backend Middleware Stack

| Order | Middleware | Type | Purpose |
|---|---|---|---|
| 1 | `cors()` | Third-party | Allow requests from CLIENT_URL. |
| 2 | `express.json()` | Built-in | Parse JSON request bodies. |
| 3 | `requestLogger` | **Custom** | Logs method, URL, status and response time of each request. |
| 4 | `express.static('public')` | Built-in | Serves logo, icons and email assets at /public. |
| 5 | `protect` | **Custom** | Verifies JWT, attaches req.user. |
| 6 | `checkProjectMember / checkProjectOwner` | **Custom** | Loads project, enforces membership/ownership, attaches req.project. |
| 7 | `validate(schema)` | **Custom** | Validates request body/query and returns 400 with field errors. |
| 8 | `notFound` | **Custom** | Returns 404 JSON for unknown routes. |
| 9 | `errorHandler` | **Custom** | Central error-to-status mapping and JSON error response. |

Routes chain middleware, for example: `router.patch('/:taskId/status', protect, loadTaskAndProject, validate(statusSchema), updateStatus)`.

## 8.2 Backend Folder Structure

```
teamflow-server/
  server.js                # starts HTTP server
  app.js                   # express app, middleware chain
  config/db.js             # mongoose connection
  models/                  # User, Project, Task, Comment, Activity, Notification
  routes/                  # auth, project, task, comment, activity, dashboard routes
  controllers/             # request handlers per module
  middleware/              # protect, projectAccess, validate, requestLogger, errorHandler
  services/                # emailService (Resend), activityService (logActivity)
  utils/                   # ApiError, asyncHandler, pagination helpers
  public/                  # static assets served by express.static()
  tests/postman/           # exported Postman collection and environment
  .env  (ignored)   .env.example   .gitignore   package.json
```

## 8.3 Frontend Folder Structure and Hooks

```
teamflow-client/src/
  main.jsx, App.jsx        # router setup
  api/                     # axios/fetch client with token header
  context/AuthContext.jsx  # useContext-based auth state
  hooks/                   # useApi (loading/error/data), useDebounce
  components/              # Navbar, ProtectedRoute, Modal, Spinner, Toast, Pagination ...
  pages/                   # Login, Register, Projects, ProjectDetails, Board,
                           # TaskDetails, ActivityFeed, ProjectDashboard, MyDashboard, NotFound
  features/                # per-module components (projects/, tasks/, collab/, dashboard/)
```

| Hook | Planned use |
|---|---|
| useState | Form fields, filters, modal visibility, pagination state. |
| useEffect | Loading data on mount or when filters change; restoring session on reload. |
| useContext | AuthContext (user, token, login, logout) shared across pages. |
| useRef | Auto-focus the title field in the task modal and the comment textarea. |
| Custom hook | useApi / useDebounce to avoid duplicated loading-error logic and to debounce search. |

## 8.4 Environment Variables

| Variable | Side | Purpose |
|---|---|---|
| PORT | Server | HTTP port. |
| NODE_ENV | Server | development or production. |
| MONGO_URI | Server | MongoDB Atlas connection string. |
| JWT_SECRET | Server | JWT signing secret. |
| JWT_EXPIRES_IN | Server | Token lifetime, e.g. 1d. |
| CLIENT_URL | Server | Allowed CORS origin and base URL for email links. |
| SERVER_PUBLIC_URL | Server | Public backend URL used to build the /public logo link in emails. |
| RESEND_API_KEY | Server | Resend API key (secret). |
| EMAIL_FROM | Server | Sender address used for emails. |
| VITE_API_BASE_URL | Client | Base URL of the backend API. |

# 9. Testing Requirements

Testing is performed by every member for their own module (frontend behaviour, backend endpoints and Postman collection). A single Postman collection with one folder per module is stored in the repository, uses environment variables `{{baseUrl}}` and `{{token}}`, and includes `pm.test` assertions on status code and response body. Both successful and unsuccessful requests are demonstrated, including authentication and invalid input.

## 9.1 Shared Core / Authentication

| ID | Scenario | Expected result |
|---|---|---|
| TC-AUTH-01 | Register with valid data | 201, user returned without password |
| TC-AUTH-02 | Register with existing email | 409 |
| TC-AUTH-03 | Register with short password / bad email | 400 with field errors |
| TC-AUTH-04 | Login with correct credentials | 200, token returned |
| TC-AUTH-05 | Login with wrong password | 401 |
| TC-AUTH-06 | Call protected route with no token / malformed / expired token | 401 |

## 9.2 Module 1: Projects (Bansari Akhani)

| ID | Scenario | Expected result |
|---|---|---|
| TC-PRJ-01 | Create project with valid data | 201, caller is owner and member |
| TC-PRJ-02 | Create project with name shorter than 3 chars or end before start | 400 |
| TC-PRJ-03 | List projects with search, sort and pagination | 200, only own projects, pagination metadata |
| TC-PRJ-04 | Get project as non-member | 403 |
| TC-PRJ-05 | Get project with unknown id / malformed id | 404 / 400 |
| TC-PRJ-06 | Update project as Member (non-owner) | 403 |
| TC-PRJ-07 | Add member with unregistered email / already-member email | 404 / 409 |
| TC-PRJ-08 | Remove the Owner | 400 |
| TC-PRJ-09 | Delete project as Owner, then GET it | 200, then 404; related tasks removed |

## 9.3 Module 2: Tasks (Heer Sachdev)

| ID | Scenario | Expected result |
|---|---|---|
| TC-TSK-01 | Create task with valid data | 201, status To Do |
| TC-TSK-02 | Create task with empty title / invalid priority / invalid date | 400 |
| TC-TSK-03 | Assign task to non-member | 400 |
| TC-TSK-04 | List tasks with status, priority, assignee, due range, search and sort combinations | 200, correct subset and order |
| TC-TSK-05 | PATCH status To Do to In Progress | 200 |
| TC-TSK-06 | PATCH status To Do to Done / same status / unknown status | 400 each |
| TC-TSK-07 | PUT task with missing required field | 400 |
| TC-TSK-08 | Task update by non-member | 403 |
| TC-TSK-09 | Delete task by member who is neither creator nor owner | 403 |

## 9.4 Module 3: Collaboration (Ishpreet Kaur)

| ID | Scenario | Expected result |
|---|---|---|
| TC-COL-01 | Add comment with valid text | 201, activity COMMENT_ADDED created |
| TC-COL-02 | Add empty or whitespace-only comment / over 1000 chars | 400 |
| TC-COL-03 | Add comment as non-member of the project | 403 |
| TC-COL-04 | Edit own comment / edit another user's comment | 200 with editedAt / 403 |
| TC-COL-05 | Delete comment as author, as Owner, as other member | 200 / 200 / 403 |
| TC-COL-06 | Task activity after create, assign and status change | Matching ordered records with actor and details |
| TC-COL-07 | Attempt PUT/DELETE on activity endpoint | 404 (no such route) |

## 9.5 Module 4: Dashboard and Notifications (Kriti Patel)

| ID | Scenario | Expected result |
|---|---|---|
| TC-DSH-01 | Summary for seeded project with known counts | Totals, status counts and completion rate match seed data |
| TC-DSH-02 | Summary for project with no tasks | 200, zeros, completion rate 0 |
| TC-DSH-03 | Workload with assigned, unassigned and overdue tasks | Correct per-member and Unassigned buckets |
| TC-DSH-04 | Overdue boundary: due yesterday vs. due tomorrow vs. Done and past due | Only not-Done past-due tasks listed |
| TC-DSH-05 | Invalid dateFrom / dateFrom later than dateTo | 400 |
| TC-DSH-06 | Assign task and check Notification record | Notification status sent (or failed with error) |
| TC-DSH-07 | Reminder on unassigned or Done task | 400 |
| TC-DSH-08 | Reminder with invalid RESEND_API_KEY | 502, Notification status failed |
| TC-DSH-09 | Reminder by a non-owner member / notification log by non-owner | 403 |

## 9.6 Frontend Verification

- Manual checklist per module: loading indicator visible, error message on failed request, success feedback after mutation, responsive layout at 360 px, 768 px and 1280 px.
- End-to-end flow demonstration: register, log in, create project, add member, create and assign task, change status, comment, view activity, view dashboard, send reminder.

# 10. Deployment and Documentation Requirements

## 10.1 Deployment

| Component | Platform | Requirements |
|---|---|---|
| Frontend | Vercel | Build with Vite; set VITE_API_BASE_URL; add SPA rewrite so React Router paths reload correctly. |
| Backend | Render (web service) | Set all server environment variables in the dashboard; start script `node server.js`; CORS set to the Vercel URL. |
| Database | MongoDB Atlas | Cluster and database user created; network access configured so the Render service can connect; URI stored only in MONGO_URI. |
| Email | Resend | API key stored only on Render; sender set in EMAIL_FROM. |

Platform choices are flexible and may change at the final setup stage; the working public URL shall be provided in the README and the report.

## 10.2 Repository and Documentation

- The project is maintained in one GitHub repository with commits by all four members.
- README.md shall contain: project description and features; installation and execution instructions; project structure; API endpoints; environment-variable requirements.
- A .env.example file lists variable names without secrets; .env is in .gitignore.
- A short report with screenshots of each module and the main workflow is submitted.
- The exported Postman collection and environment are stored in the repository.

## 10.3 Individual Contribution Matrix

Each member contributes to frontend, backend and testing of their own module (as required for the individual viva).

| Member | Frontend | Backend | Testing | Viva demonstration focus |
|---|---|---|---|---|
| Bansari Akhani | Projects pages, member management UI | Project/membership APIs, project access middleware, user search | TC-PRJ series | Membership checks, PUT/DELETE, route and query params |
| Heer Sachdev | Kanban board, task forms, filters | Task APIs, status transition rules, filtering and sorting | TC-TSK series | PATCH vs PUT, validation, task lifecycle |
| Ishpreet Kaur | Comment thread, activity timeline | Comment/activity APIs, logActivity service, task-based access middleware | TC-COL series | References between collections, middleware chaining, immutable history |
| Kriti Patel | Dashboard, charts, reminder UI | Aggregation endpoints, Resend email service, notification log | TC-DSH series | Aggregation pipelines, third-party API handling and failures |

# 11. Traceability to IT645 Course Guidelines

| Guideline | Requirement | Where addressed in this SRS |
|---|---|---|
| A | Frontend, backend, database integration and full working flow | 3.x, 5, 9.6 |
| B-a | Semantic HTML | 4.1, NFR-06 |
| B-b | Responsive web design | 4.1, NFR-05 |
| B-c | Tailwind CSS | 2.5, 4.1 |
| B-d | React functional components | 2.5, 8.3, FR-COM-03 |
| B-e | Hooks: useState, useEffect, useRef, useContext | 8.3, FR-AUTH-07, FR-TSK-10, FR-COL-09 |
| B-f | React Router | 4.1 |
| B-g | fetch()/axios for HTTP requests | 4.3, 8.3 |
| B-h | Loading, success and error states | 4.1, each Frontend FR |
| B-i | Meaningful third-party Web API | FR-DSH-06 to FR-DSH-10, 4.3 (Resend; called through the backend, triggered and displayed from the UI) |
| C-a | Clear endpoints, route parameters, query parameters | 5.2 to 5.5 |
| C-b | GET, POST, PUT, DELETE, PATCH | 5.6 |
| C-c | Proper status codes | 5.7 |
| C-d, C-n | Express middleware including custom middleware and chaining | 8.1 |
| C-e | JWT authentication and protected routes | FR-AUTH-03 to FR-AUTH-05 |
| C-f, C-m | express.json(), express.static(), CORS, logging | 8.1, 4.4 |
| C-g | API error handling | FR-COM-02, 5.7 |
| C-h, C-o | Secrets in .env and .gitignore | 2.5, 8.4, NFR-02 |
| C-i, C-j | Modules (routes, controllers, models, middleware); schemas in /models | 8.2, 6 |
| C-k, C-l | MongoDB/Mongoose persistence and CRUD | 6, 5 |
| D-a, D-b, D-c | Postman tests: success, failure, auth, invalid input | 9 |
| E-a to E-e | Modular structure, ES6+, naming, reuse | 2.5, NFR-07, 8.2, 8.3 |
| E-f | GitHub repository | 10.2 |
| E-g | README contents | 10.2 |
| E-h | Short report with screenshots | 10.2 |
| E-i | Deployment with working URL | 10.1 |
| Team rule | One major module per member with frontend, backend and testing each; independent evaluation | 3.2 to 3.5, 10.3 |

# Appendix A: Design Decisions for Team Confirmation

The following specifics were not stated in the course guidelines or by the team and are proposed here so the SRS is implementable. Please confirm or change each item before the SRS is finalised.

| ID | Decision proposed in this SRS | Alternative |
|---|---|---|
| D1 | React project is built with Vite. | Create React App or Next.js. |
| D2 | JWT lifetime 1 day; token stored in browser localStorage and sent as Bearer header. | Shorter lifetime, or httpOnly cookie. |
| D3 | Status transitions limited to To Do - In Progress - Done with reopen from Done to In Progress. | Allow any status change. |
| D4 | Owner: edit/delete project, manage members, delete any task/comment, send reminders. Members: create/edit tasks, change status and assignee, comment. Task deletion by Owner or creator. | Different permission split. |
| D5 | Overdue = due date earlier than current time and status not Done. | Compare by calendar day only. |
| D6 | Dashboard charts drawn with a client-side library such as Recharts. | Tailwind-only bars or Chart.js. |
| D7 | Past due dates are accepted when creating or editing a task. | Reject past dates. |
| D8 | Removing a member unassigns their tasks; deleting a project cascades to all related data. | Block removal while tasks are assigned. |
| D9 | Node.js LTS v18 or later. | Specific version chosen by team. |
| D10 | The Resend API is called by the backend (to protect the key). The frontend triggers it (assignment, Send reminder) and displays the result. | Confirm with the instructor that backend invocation satisfies guideline B-i, which is listed under the frontend. |
| D11 | Resend's test sender can deliver only to the account owner's address until a domain is verified. Demo plan: use team members' addresses only if a domain is verified; otherwise demonstrate with the account owner's address and show the Notification log. | Verify a domain, or use a different email provider. |
| D12 | Professor's name and final submission date are placeholders on the cover page. | Fill in before submission. |

End of document.